CRUD Es un acrónimo que representa las 4 operaciones básicas que puedes hacer con la información en una base de datos: (créate, red, update, delete.)

El CRUD sirve para que el texto que extraen de los archivos no se pierda al cerrar el programa. "Persistir" significa que los datos se quedan guardados en el disco (en su caso, usar MongoDB, una base de datos no relacional). Para trabajar con el CRUD de documentos persistidos, se necesita tener donde guardar esos datos. En este caso se usa MongoDB.

En tu código, esto se suele dividir en tres capas según la arquitectura que están usando (Clean Architecture):

Repositorio (repositories): Es donde escribes el código que "habla" directamente con la base de datos (usando motor o beanie que vi en tu pyproject.toml).

Ejemplo: db.collection.insert_one(documento)

Servicio (services): Es donde pones la lógica. Por ejemplo, antes de "Crear", verificar que el PDF no esté vacío.

Esquema (schemas): Defines qué datos tiene un "Documento" (ID, nombre del archivo, texto extraído, fecha).

instalación mongoDB

Pasos:

1. instalar mongodb
   Para que tu código Python pueda "persistir" los documentos, necesitas el motor de la base de datos:
   a. **Descarga MongoDB Community Server:** Ve a la página oficial y baja el instalador ".msi".
   b. **Instalación:** Dale a "Next" a todo. Asegúrate de que la casilla **"Install MongoDB as a Service"** esté marcada.
   c. **MongoDB Compass:** El instalador te preguntará si quieres instalar "Compass". Di que **sí**. Es una interfaz gráfica (como una ventana de Windows) para ver tus datos sin usar la terminal.

   Tutorial: https://youtu.be/RNWdpWGNULw?si=AxzQhORZQL3pDjca

2. Conectar VS Code con MongoDB

   a. Ve al icono de **Extensiones** en la barra lateral izquierda (el que parecen 4 cuadraditos).
   b. Busca e instala **"MongoDB for VS Code"**.

c. Haz clic en el nuevo icono de una hojita con el logo de MongoDB que aparecerá a la izquierda y dale a **"Add Connection"**. Generalmente, la ruta por defecto es mongodb://`localhost:27017`.

3. ¿Cómo se ve el código del CRUD?

a. Como se trabaja en el proyecto pdf-extraxtext, la tarea es crear las funciones que guarden el texto. En tu archivo database.py o en la carpeta repositories, el código para el "Create" (Crear) se verá algo así:

```python
async def crear_documento(db, contenido_texto):
    nuevo_doc = {
        "filename": "archivo.pdf",
        "content": contenido_texto,
        "date": "2026-04-18"
    }
    # Esto guarda el documento en la colección 'documentos'
    resultado = await db.documentos.insert_one(nuevo_doc)
    return resultado.inserted_id
```

b. **Read (Leer / Buscar)**

Esta operación sirve para obtener la información de un usuario específico de la base de datos usando su identificador o nombre de usuario.

```python
async def obtener_usuario(db, usuario_id):
    # Buscamos un documento cuyo "_id" coincida con el que recibimos
    usuario = await db.usuarios.find_one({"_id": usuario_id})

    if usuario:
        print(f"Usuario encontrado: {usuario['nombre']}")
        return usuario
    else:
        print("Usuario no encontrado.")
        return None
```

c. **Update (Actualizar / Modificar)**

Se usa cuando el usuario quiere cambiar algún dato, por ejemplo, actualizar su correo electrónico o su nombre.

```python
async def actualizar_correo_usuario(db, usuario_id, nuevo_correo):
    # 'update_one' busca por ID y '$set' indica qué campos queremos cambiar
    resultado = await db.usuarios.update_one(
        {"_id": usuario_id},
        {"$set": {
            "correo": nuevo_correo,
            "fecha_actualizacion": "2026-04-19" # Fecha opcional de seguimiento
        }}
    )

    if resultado.modified_count > 0:
        return "Correo actualizado con éxito"
    return "No se realizaron cambios"
```

d. Delete (Borrar / Eliminar)

Esta operación elimina permanentemente el documento de la colección. (En **MongoDB**, una **colección** es el equivalente a una **tabla** en las bases de datos tradicionales (como SQL). Básicamente, es el nombre del grupo de datos sobre el que quieres aplicar la operación. Por ejemplo, podrías tener una colección de usuarios y otra de documentos_pdf.)

```python
async def eliminar_usuario(db, usuario_id):
    # 'delete_one' elimina el primer documento que coincida con el filtro
    resultado = await db.usuarios.delete_one({"_id": usuario_id})

    if resultado.deleted_count > 0:
        return "Usuario eliminado de la base de datos"
    return "No se encontró el usuario para eliminar"
```

Resumen del Esquema de Datos

Para que estas funciones trabajen bien, el documento en tu **MongoDB** se vería así (similar al JSON de la "Opción 1" de tu imagen):

```JSON
{
  "_id": "usr_99",
  "nombre": "Pablo Condori",
  "user": "pablo_dev",
  "correo": "pablo.ejemplo@mail.com",
  "fecha": "2026-04-19"
}
```

¿Cómo encaja esto en el proyecto?

Puedes aplicar esta misma lógica de arriba pero cambiando la colección usuarios por documentos y ajustando los campos para que guarden el texto extraído del PDF.

4. verificar

Verificar que el servicio de MongoDB esté corriendo.

Ejecutar el comando uv run pytest.

Ojo: Probablemente los tests de "Repository" ahora te den error porque no encuentran la base de datos conectada. Eso es bueno, ¡es parte del ciclo RED del TDD!

En tu aplicación, el CRUD es el conjunto de rutas (endpoints) que permitirán gestionar los PDFs ya procesados en MongoDB.

DATOS

CRUD debe cubrir estos 4 pilares:

1. No puedes crear un CRUD si no sabes qué datos vas a guardar. Documenta que un "Documento" en la base de datos debería tener, como mínimo:

ID: Único (generado por MongoDB).

Filename: Nombre original del PDF.

Content: El texto puro que extrajiste.

Checksum: El código SHA-256 (para que no haya duplicados).

Size: Tamaño del archivo en bytes (para validar que no sea muy pesado).

2. Especificación de los Endpoints (FastAPI)

Documenta cómo se verán las "puertas de entrada" a tu aplicación. Esto se llama API Contract:

POST /documents: Recibe el PDF, lo procesa y lo guarda.

GET /documents: Devuelve una lista de todos los documentos guardados (solo nombres e IDs, sin el texto gigante para no saturar).

GET /documents/{id}: Devuelve todo el detalle de un PDF específico, incluyendo su texto.

DELETE /documents/{id}: Borra el registro de la base de datos.

3. Lógica de Negocio (Las Reglas de Oro)

Aquí es donde aplicas los requisitos que te pasaron. Debes documentar que el flujo será:

Validar: ¿Es un PDF? ¿Pesa menos de X MB?

Checksum: Calcular el hash del archivo. Si ya existe en la DB, abortar (Error 409 Conflict).

Extracción: Sacar el texto usando una librería (investiga PyMuPDF o pdfplumber).

Guardado: Persistir en MongoDB de forma asíncrona.

4. Tecnologías a utilizar (Stack Técnico)

Para tu documentación, menciona estas herramientas:

Motor / Beanie: Para conectar Python con MongoDB de forma asíncrona.

Pydantic: Para validar que los datos que entran y salen de la API sean correctos.

Hashlib: Librería nativa de Python para calcular el Checksum.

Se propone utilizar el algoritmo **SHA-256** mediante la librería nativa hashlib. El checksum se almacenará en un campo indexado de MongoDB para garantizar que la búsqueda de duplicados sea eficiente antes de cada inserción. Es más seguro y evita "colisiones" (cuando dos archivos distintos dan el mismo hash, algo que en MD5 puede pasar).

Python

```python
import hashlib

def calcular_checksum(contenido_archivo: bytes):
    # Creamos un objeto hash usando el algoritmo SHA-256
    sha256_hash = hashlib.sha256()

    # "Leemos" el contenido para generar la huella
    sha256_hash.update(contenido_archivo)

    # Devolvemos la cadena de texto final (el hash)
    return sha256_hash.hexdigest()
```

Esa es la lógica perfecta para cumplir con el requisito de **"no duplicados"**.

Los archivos en mongodb se guardan en filay y/o columnas

En **SQL** tenés **tablas → filas → columnas**.
En **MongoDB** la lógica cambia a:

- **Base de datos**
- **Colecciones (collections)** → equivalente a tablas
- **Documentos (documents)** → equivalente a filas
- **Campos (fields)** → equivalente a columnas

Pero la diferencia clave es que **los documentos son JSON (más precisamente BSON)**, o sea estructuras flexibles.

Guardar datos

En vez de esto en SQL:

```sql
Tabla: usuarios
id | nombre | email
```

En MongoDB sería una colección `usuarios` con documentos así:

```json
{
  "_id": "abc123",
  "nombre": "Lautaro",
  "email": "lautaro@mail.com"
}
```

Para el pdf mas el texto extraído hay dos opciones:

1.

```json
{
  "_id": "doc1",
  "usuario_id": "user123",
  "nombre_archivo": "archivo.pdf",
  "texto": "Todo el texto extraído del PDF...",
  "fecha": "2026-04-19"
}
```

- Cada documento está **asociado a un usuario**
- No es público, vos controlas quién lo ve.

2. MongoDB permite guardar archivos con **GridFS**, pero:

- Es más complejo
- Generalmente **no es necesario** si solo querés el texto

Alternativa más usada:

- Guardás el PDF en disco o cloud (ej: AWS S3)
- En MongoDB guardás solo la referencia:

Para que el usuario vea solo lo que ha hecho al hacer el login tiene que tener su id

1. Tenés su `user_id`
2. Cuando guardás:

JSON

```
{
  "usuario_id": "123",
  ...
}
```

3. Cuando consultás:

JavaScript

```
db.documentos.find({ usuario_id: "123" })
```

Vos vas a usar una colección tipo documentos.

Cada documento debe tener usuario_id.

Filtrás por ese campo para privacidad.

Dato extra: Un **hash** es como una "huella digital" única para un conjunto de datos. Es el resultado de pasar cualquier tipo de información (un texto, un archivo, una imagen) por un algoritmo matemático que lo transforma en una cadena de caracteres de longitud fija.